

Relatório Parcial: Preparação do Dataset WoodScape e Treinamento Inicial com RT-DETR

Matheus Higino

11 de setembro de 2026

Resumo

Este relatório documenta as etapas realizadas até o momento para preparação do dataset WoodScape e execução de um treinamento inicial utilizando a arquitetura RT-DETR por meio da biblioteca Ultralytics. O objetivo desta etapa foi construir e validar um pipeline reprodutível para detecção de objetos em imagens fisheye, contemplando a obtenção e organização dos dados, conversão das anotações para o formato utilizado pela Ultralytics, divisão do conjunto em treinamento, validação e teste, configuração do ambiente computacional e execução de um treinamento preliminar de três épocas. O procedimento resultou em 8.234 imagens válidas, distribuídas entre treinamento, validação e teste, e permitiu confirmar o funcionamento do pipeline completo em GPU. Os resultados apresentados devem ser interpretados como validação técnica do processo, e não como desempenho final do modelo.

1 Introdução

O WoodScape é um dataset voltado à percepção visual automotiva utilizando câmeras fisheye, disponibilizado pela Valeo para diferentes tarefas de visão computacional, incluindo segmentação semântica, segmentação por instância e detecção de objetos por meio de *bounding boxes* 2D [1, 2].

Neste trabalho, o WoodScape está sendo utilizado para investigar técnicas de detecção de objetos em imagens fisheye. Nesta primeira etapa, foi escolhido o modelo RT-DETR (*Real-Time DEtection TRansformer*), uma arquitetura de detecção end-to-end baseada em Transformers e projetada para combinar boa precisão e baixa latência [3]. A implementação utilizada corresponde à interface disponibilizada pela biblioteca Ultralytics [4].

2 Objetivo desta etapa

O objetivo desta etapa foi construir e verificar um pipeline completo que permitisse:

- a) organizar os dados originais do WoodScape;
- b) converter as anotações de detecção para o formato esperado pela Ultralytics;

- c) dividir o dataset em treinamento, validação e teste;
- d) configurar o RT-DETR-L utilizando pesos pré-treinados;
- e) executar o treinamento utilizando GPU;
- f) verificar perdas, métricas e tempo de inferência;
- g) identificar eventuais problemas nas anotações e na configuração antes do treinamento definitivo.

3 Dataset WoodScape

3.1 Dados utilizados

Foram utilizados os dados RGB e as anotações de *bounding boxes* 2D do WoodScape. O repositório oficial informa que o conjunto público disponibilizado contém aproximadamente 8,2 mil imagens, além de amostras originalmente reservadas para benchmark, e oferece caixas 2D para classes relacionadas a veículos, pedestres, bicicletas, semáforos e placas de trânsito [2].

Após a preparação dos arquivos, obteve-se:

Tabela 1: Arquivos disponíveis antes da conversão.

Tipo	Quantidade
Imagens RGB	8.234
Arquivos de anotação	8.234
Pares válidos imagem/anotação	8.234

A estrutura utilizada para os dados originais foi:

```
datasets/
  woodscape_raw/
    rgb_images/
    box_2d_annotations/
```

As imagens apresentam identificadores que incluem a posição da câmera, por exemplo:

```
04346_FV.png
07370_RV.png
06208_MVL.png
03110_MVR.png
```

3.2 Formato original das anotações

As anotações originais foram encontradas no seguinte formato:

```
vehicles,0,11,387,250,570
vehicles,0,1005,358,1243,522
person,1,865,295,915,375
```

Cada linha segue a estrutura:

$$\text{classe, id, } x_{\min}, y_{\min}, x_{\max}, y_{\max}. \quad (1)$$

As cinco classes utilizadas foram:

Tabela 2: Mapeamento das classes utilizadas no treinamento.

ID	Classe
0	vehicles
1	person
2	bicycle
3	traffic_light
4	traffic_sign

4 Conversão das anotações

A Ultralytics utiliza, para detecção de objetos, anotações normalizadas no formato:

$$\text{class_id } x_c \ y_c \ w \ h, \quad (2)$$

em que x_c e y_c correspondem ao centro da caixa e w e h correspondem à largura e altura da caixa.

Considerando uma imagem de largura W e altura H , a conversão foi realizada por:

$$x_c = \frac{x_{\min} + x_{\max}}{2W}, \quad (3)$$

$$y_c = \frac{y_{\min} + y_{\max}}{2H}, \quad (4)$$

$$w = \frac{x_{\max} - x_{\min}}{W}, \quad (5)$$

$$h = \frac{y_{\max} - y_{\min}}{H}. \quad (6)$$

Foi desenvolvido o script:

```
scripts/prepare_woodscape.py
```

executado por meio do comando:

```
python3 prepare_woodscape.py \
  --raw ../datasets/woodscape_raw \
  --output ../datasets/woodscape
```

5 Divisão do dataset

O script de preparação criou automaticamente as divisões de treinamento, validação e teste.

A estrutura final passou a ser:

Tabela 3: Distribuição do WoodScape após preparação.

Conjunto	Imagens	Objetos	Proporção aprox.
Treinamento	6.587	57.719	80%
Validação	823	7.118	10%
Teste	824	7.226	10%
Total	8.234	72.063	100%

```

datasets/
  woodscape/
    images/
      train/
      val/
      test/
    labels/
      train/
      val/
      test/
  woodscape.yaml

```

5.1 Estratégia de separação

Durante a preparação, as imagens foram agrupadas considerando o identificador da captura e a posição da câmera. O objetivo foi reduzir o risco de colocar imagens fortemente relacionadas de uma mesma captura em conjuntos diferentes. Imagens associadas aos sufixos FV, RV, MVL e MVR permanecem na mesma partição quando compartilham o mesmo identificador principal.

6 Ambiente computacional

O treinamento inicial foi realizado em uma estação Alienware equipada com GPU NVIDIA GeForce RTX 5070.

Tabela 4: Ambiente utilizado no treinamento inicial.

Componente	Configuração
GPU	NVIDIA GeForce RTX 5070
VRAM reportada pelo PyTorch	11,49 GB
Driver NVIDIA	595.84
CUDA reportado pelo driver	13.2
CUDA utilizado pelo PyTorch	13.0
Python	3.14.4
PyTorch	2.14.0+cu130
Ultralytics	8.4.147

7 Configuração do RT-DETR

Foi utilizada a variante RT-DETR-L, carregada a partir do arquivo pré-treinado `rtdetr-1.pt`. A Ultralytics oferece suporte ao RT-DETR para treinamento, validação, inferência e exportação [4]. O modelo original RT-DETR foi proposto como detector end-to-end em tempo real baseado em Transformer [3].

Tabela 5: Principais parâmetros do treinamento preliminar.

Parâmetro	Valor
Modelo	RT-DETR-L
Épocas	3
Tamanho da imagem	640
Batch size	4
Device	CUDA:0
Workers	8
Seed	42
Pretrained	verdadeiro
Deterministic	falso
AMP	verdadeiro
Otimizador	AdamW, selecionado automaticamente
Learning rate selecionado	0,001111
Momentum	0,9
Weight decay	0,0005
Warm-up	3 épocas

A opção `deterministic=False` é coerente com a implementação RT-DETR da Ultralytics, cuja documentação informa que a operação `F.grid_sample` utilizada pelo modelo não oferece suporte adequado ao modo determinístico em CUDA [5].

Durante a inicialização, o modelo adaptou o número de classes de 80 para 5 e o log indicou remapeamento de duas das cinco linhas do cabeçalho de classificação pré-treinado por nome de classe.

Após a adaptação, o modelo apresentou aproximadamente 32,8 milhões de parâmetros durante o treinamento e 109,9 GFLOPs. Na validação com o modelo fundido foram reportados aproximadamente 32,0 milhões de parâmetros e 105,4 GFLOPs.

8 Treinamento preliminar

O objetivo do primeiro treinamento foi realizar um *smoke test*, isto é, verificar se o pipeline completo funcionava corretamente antes da execução de experimentos longos.

Foram verificadas as etapas de leitura de imagens, leitura das anotações, criação do *DataLoader*, utilização da GPU, cálculo das perdas, validação e salvamento dos pesos `best.pt` e `last.pt`.

O treinamento das três épocas levou aproximadamente 0,138 horas, equivalente a cerca de 8,3 minutos.

9 Resultados preliminares

9.1 Evolução durante as três épocas

Tabela 6: Resultados obtidos ao final de cada época.

Época	Precision	Recall	mAP@50	mAP@50:95	GPU mem.
1	0,574	0,471	0,492	0,295	3,64 GB
2	0,519	0,545	0,506	0,308	3,83 GB
3	0,601	0,554	0,565	0,348	3,83 GB

Após o treinamento, o melhor modelo foi validado novamente:

Tabela 7: Resultados globais do melhor modelo após o *smoke test*.

Métrica	Resultado
Precision	0,604
Recall	0,552
mAP@50	0,564
mAP@50:95	0,347
Tempo de pré-processamento	0,1 ms/imagem
Tempo de inferência	5,5 ms/imagem
Tempo de pós-processamento	0,1 ms/imagem

9.2 Resultados por classe

Na validação final foram utilizadas 823 imagens e 7.117 instâncias efetivas.

Tabela 8: Resultados preliminares por classe no conjunto de validação.

Classe	Imagens	Instâncias	P	R	mAP@50	mAP@50:95
vehicles	794	4.413	0,674	0,762	0,765	0,559
person	555	1.596	0,696	0,639	0,683	0,391
bicycle	329	715	0,507	0,413	0,430	0,242
traffic_light	69	138	0,651	0,384	0,425	0,222
traffic_sign	181	255	0,491	0,561	0,518	0,324

Mesmo em apenas três épocas, a classe **vehicles** apresentou desempenho superior às classes menos representadas, sugerindo que o desbalanceamento entre classes deverá ser considerado em análises futuras.

10 Observações sobre qualidade dos dados

Durante a leitura das anotações, a Ultralytics encontrou dois casos de rótulos duplicados:

```
train/.../03404_RV.png: 1 duplicate labels removed
val/.../01866_MVL.png: 1 duplicate labels removed
```

O script de preparação contabilizou inicialmente 7.118 objetos no conjunto de validação, enquanto o *DataLoader* utilizou 7.117 instâncias após remoção automática de uma duplicata. Recomenda-se ajustar o script `prepare_woodscape.py` para eliminar linhas duplicadas ainda na etapa de conversão.

11 Interpretação dos resultados

Os valores obtidos neste experimento não devem ser tratados como resultados finais da pesquisa. O principal motivo é que o treinamento foi realizado por apenas três épocas e a configuração utilizada possuía `warmup_epochs=3.0`. Consequentemente, praticamente todo o experimento ocorreu durante o período de *warm-up* do otimizador.

A finalidade deste treinamento foi comprovar que o dataset pode ser carregado integralmente, as anotações estão no formato adequado, o RT-DETR aprende a partir das imagens, o treinamento CUDA funciona corretamente, as métricas são calculadas e os pesos são salvos corretamente.

Assim, os resultados desta seção devem ser classificados como **resultados preliminares de validação do pipeline**.

12 Uso dos conjuntos de validação e teste

O conjunto de validação deve continuar sendo utilizado durante o desenvolvimento para acompanhar métricas e selecionar o melhor *checkpoint*. O conjunto de teste, composto por 824 imagens, deve ser preservado para a avaliação final das configurações selecionadas.

13 Próximas etapas

1. remover anotações duplicadas durante a preparação do dataset;
2. executar novamente a preparação para garantir um conjunto limpo;
3. definir explicitamente os hiperparâmetros do experimento;
4. aumentar o *batch size* de acordo com a memória disponível na RTX 5070;
5. executar treinamento completo do RT-DETR-L;
6. acompanhar curvas de perda, Precision, Recall e mAP;
7. avaliar o modelo selecionado no conjunto de teste apenas após definição da configuração;
8. utilizar exatamente os mesmos *splits* para modelos concorrentes;
9. comparar RT-DETR com arquiteturas YOLO utilizando as mesmas imagens e métricas;

10. registrar mAP@50, mAP@50:95, Precision, Recall, parâmetros, GFLOPs, memória utilizada, latência e FPS;
11. posteriormente avaliar exportação e inferência em ambientes de borda.

14 Conclusão

Foi concluída com sucesso a primeira etapa de preparação e validação do pipeline de detecção de objetos utilizando o WoodScape e o RT-DETR.

O dataset foi organizado e convertido para o formato utilizado pela Ultralytics, produzindo 8.234 pares válidos de imagem e anotação e 72.063 objetos inicialmente contabilizados. O conjunto foi dividido em treinamento, validação e teste seguindo aproximadamente uma proporção 80/10/10.

O treinamento preliminar confirmou o funcionamento integral do pipeline na GPU NVIDIA GeForce RTX 5070. Após três épocas, o modelo apresentou mAP@50 de 0,564 e mAP@50:95 de 0,347 no conjunto de validação. Entretanto, esses resultados não devem ser considerados desempenho final, pois todo o treinamento coincidiu praticamente com o período de *warm-up*.

O principal resultado desta etapa é, portanto, a obtenção de uma infraestrutura experimental funcional e reprodutível, que poderá ser utilizada tanto no treinamento definitivo do RT-DETR quanto em comparações futuras com outras arquiteturas de detecção de objetos.

A Comandos principais utilizados

A.1 Preparação do dataset

```
cd ~/Downloads/fisheye-object-detection/scripts
python3 prepare_woodscape.py \
    --raw ../datasets/woodscape_raw \
    --output ../datasets/woodscape
```

A.2 Treinamento inicial

```
cd ~/Downloads/fisheye-object-detection
python3 training/train_rtdetr.py
```

A.3 Verificação da GPU

```
nvidia-smi
```

B Arquivos gerados pelo treinamento

O treinamento preliminar armazenou seus resultados em:

```
runs/rtdetr/woodscape_test/
```

incluindo, entre outros:

```
weights/best.pt  
weights/last.pt  
labels.jpg  
results.png
```

Referências

- [1] YOGAMANI, S. et al. **WoodScape: A Multi-Task, Multi-Camera Fisheye Dataset for Autonomous Driving**. In: IEEE/CVF International Conference on Computer Vision (ICCV), 2019.
- [2] VALEO AI. **WoodScape: A multi-task, multi-camera fisheye dataset for autonomous driving**. Repositório oficial. Disponível em: <https://github.com/valeoai/WoodScape>. Acesso em: 11 set. 2026.
- [3] ZHAO, Y.; LV, W.; XU, S.; WEI, J.; WANG, G.; DANG, Q.; LIU, Y.; CHEN, J. **DETRs Beat YOLOs on Real-time Object Detection**. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, p. 16965–16974.
- [4] ULTRALYTICS. **RT-DETR Real-Time Detection Transformer**. Disponível em: <https://docs.ultralytics.com/models/rtdetr/>. Acesso em: 11 set. 2026.
- [5] ULTRALYTICS. **RTDETRTrainer – Ultralytics API Reference**. Disponível em: <https://docs.ultralytics.com/reference/models/rtdetr/train/>. Acesso em: 11 set. 2026.